MagicPay Keyboard Integration

This integration is divided into 3 modules:

- Core
- Android SDK
- iOS SDK

Android: SDK Implementation

SDK Implementation

Requirements

- Min Android SDK Version: 24+

Implementation

This SDK is hosted on Github Packages (Maven). To add it to your project, you'll need to configure your Gradle files to authenticate and access the package.

1. Configure the Github Packages Maven repository in your ***settings.gradle***

```kotlin
//settings.gradle

maven {
    url = uri("https://maven.pkg.github.com/....")
    credentials {
        username = project.findProperty("gpr.user") as String? ?:
System.getenv("USERNAME")
        password = project.findProperty("gpr.key") as String? ?:
System.getenv("TOKEN")
    }
}
```

    a. The ***gpr.user*** and ***gpr.key*** values are defined in your project's ***local.properties*** file. **These should never be committed to your repository.**
    b. If you're using CI/CD pipeline, set these values as environment variables (e.g. ***USERNAME*** and ***TOKEN***) in your pipeline configuration.
2. Add your personal access token
    a. Make sure to set your GitHub username as ***gpr.user*** and your personal access token as ***gpr.key***.

```kotlin
//local.properties

  github.token=YOUR_TOKEN
  github.actor=digital
```

3. Add the SDK dependency:

```kotlin
//build.gradle (module)

dependencies{
	implementation("com.magic:-magic:xxxxxx")
}
```

4. For detailed instructions on how to work with Github Packages and Gradle, refer to the official documentation (doc for Kotlin DSL and Groovy): [Working with the Gradle registry - Github Docs]

Note: [How to create a token]

Initialize

Before using the SDK features, you must set the following data to initialize

```kotlin
import com.magic.payments.magic.EnvironmentData
import com.magic.payments.magic.MagicManager
import com.magic.payments.magic.MagicEnvironmentType

class MyApp: Application{
	override fun onCreate(){
		super.onCreate()

		val myEnvironmentData = EnvironmentData(
			type = MagicEnvironmentType.DEVELOPMENT
			apiKey = "YOUR_API_KEY"
		)

		MagicManager.init(
			applicationContext = this,
			deviceId = "YOUR_DEVICE_ID", //It's nullable
			environmentData = myEnvironmentData
		)
	}
}
```

If you don't have a device ID, you can set it to null, we'll generate a device ID for you.

The options for EnvironmentType are `MagicEnvironmentType.PRODUCTION` and `MagicEnvironmentType.DEVELOPMENT`

Features Implementation

There are two types of features: **application** and **keyboard**.
All features are built using Jetpack Compose, the functions return composables and you have to add in your ***NavHost*** or else add the composable function in a fragment.

1. Keyboard Features

You don't need to manually add the keyboard to your project. It's declared in the SDK Manifest and included automatically at compile time.

To use the keyboard, the user must first enroll.

2. Application Features

The Features Composable takes care of checking if the user has already completed onboarding. Your only responsibility is to display our flows when needed. If the user has already completed onboarding, they will be taken to the Magic home screen.

Jetpack Compose Integration

```kotlin
NavHost(
	navController = navController,
	startDestination = "YOUR_START_DESTINATION_ROUTE"
){
	composable("MAGIC_FEATURES_ROUTE"){
		MagicManager.App.Magic(
			onUserDataNeeded = {
				userData
			}
		)
	}
}
```

The `onUserDataNeeded` lambda requires a `UserData` object.
You can retrieve this information from anywhere, it is recommended to load the data before displaying the Magic features.
This data is only collected after the user accepts the terms and conditions, and it will be securely stored afterward.

UserData and AuthorizationData Example

Kotlin

```kotlin
import com.magic.payments.magic.AccountData
import com.magic.payments.magic.ClientAuthorizationData
import com.magic.payments.magic.MagicManager
import com.magic.payments.magic.UserData

//There should be at least one object in the list
val accountList = listOf(
       AccountData(
              accountId = "ACCOUNT_ID",
              alias = "ACCOUNT_ALIAS"//This is visible to the user in the app
       )
)

val authorizationData = ClientAuthorizationData(
       password = "PASSWORD", //nullable
       pin = "PIN", //nullable
       base64PublicKey = "" //required
)


val userData = UserData(
       userId = "USER_ID",
       username = "USERNAME",
       accountList = accountList,
       authorizationData = authorizationData
)
```

Details:

- **password**: The user password for login, it will be encrypted with asymmetric key (base64PublicKey) and thenre-encrypted with the KeyStore using biometrics.
  If not provided, the user will be required to enter their password during Magic Onboarding.
- **pin**: If your app requires users to authorize transactions using a PIN stored securely with biometrics, you can pass it to this parameter.
  The SDK will encrypt and store it allowing biometric authentication during payments.
  If not provided, the user will need to enter the PIN manually on each payment.
- **base64PublicKey**: Required to encrypt password and pin.
  This parameter expects a PublicKey object encoded in Base64 format.

Non-Compose Integration

if you're not using Jetpack Compose yet, here is how to integrate using a Fragment:

Kotlin

```kotlin
class MiFragment : Fragment() {

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        return ComposeView(requireContext()).apply {
            setContent {
                MagicManager.App.Magic(
                    onUserDataNeeded = {
                           userData
                    }
                )
            }
        }
    }
}
```

iOS: SDK Implementation v2

Magic Pay Keyboard Integration - iOS SDK Guide

Minimum iOS Version: 16+
Components: Main SDK + Magic Keyboard Extension

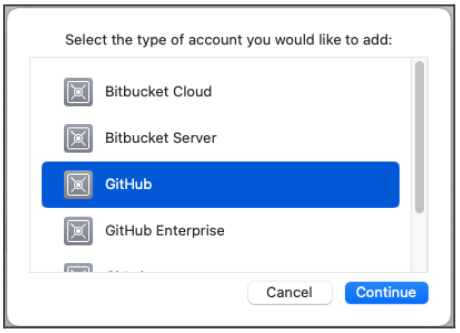
1. Configure GitHub Source Control in Xcode

This allows Xcode to download the SDK from the private repository.

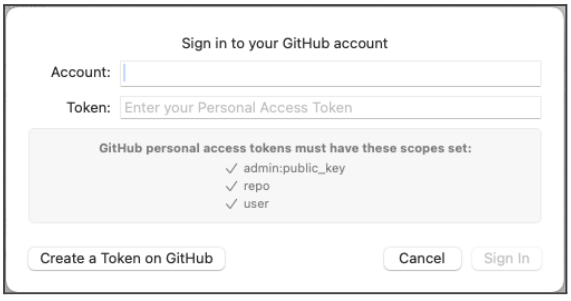
1. Open **Xcode → Settings → Source Control**

2. Click **Add Account…**

3. Choose **GitHub**

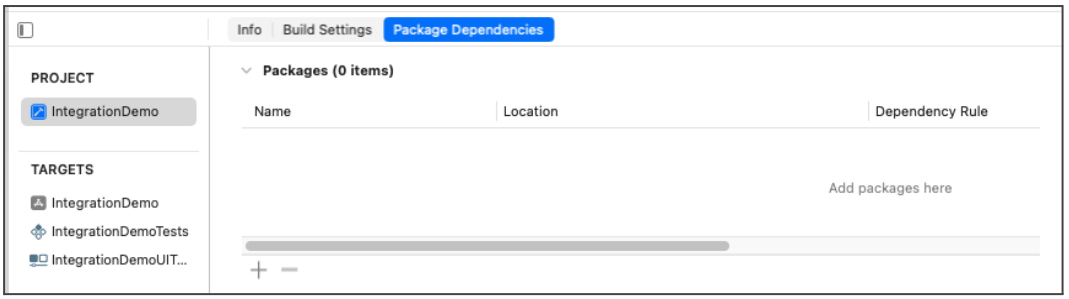


4. Sign in using a **Personal Access Token (PAT)**
   (Generate one at: GitHub → Settings → Developer Settings → Personal Access Tokens → Fine-grained PAT)



2. Add the Magic SDK to the App (Main Target)

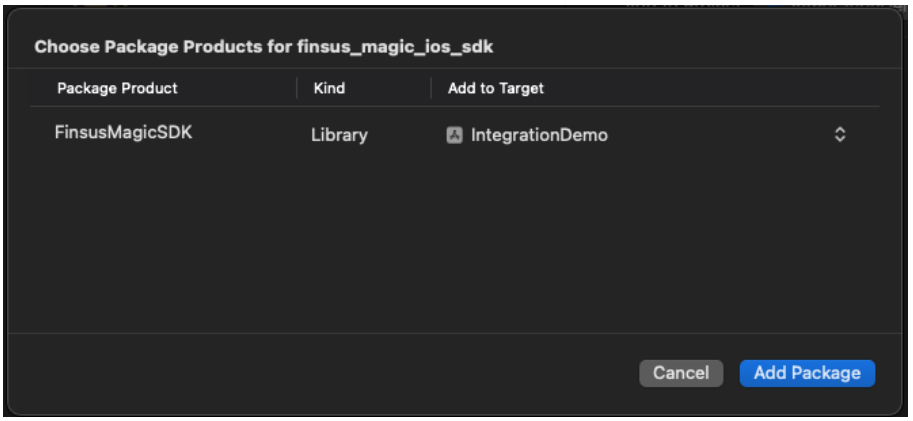
1. Select your **Project**
2. Open **Package Dependencies**
3. Click **+**



4. Select **Add Package from GitHub**
5. Enter the repository URL:

```
None
https://github.com/.....
```

6. **Dependency Rule:** Exact Version → `0.2.2`
7. **Add to Target:** Select your **Main App** target



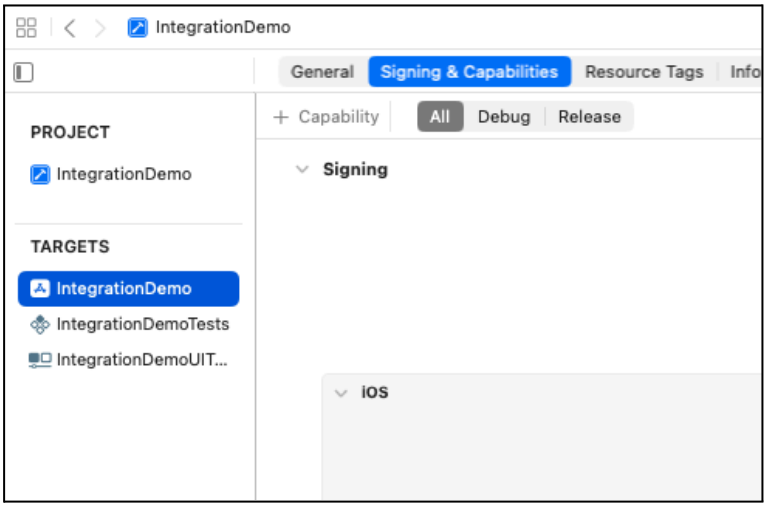
8. Click **Add Package**

```
None
Importante: Más adelante, cuando el teclado esté creado, asignaremos el
mismo paquete al target del teclado.
```

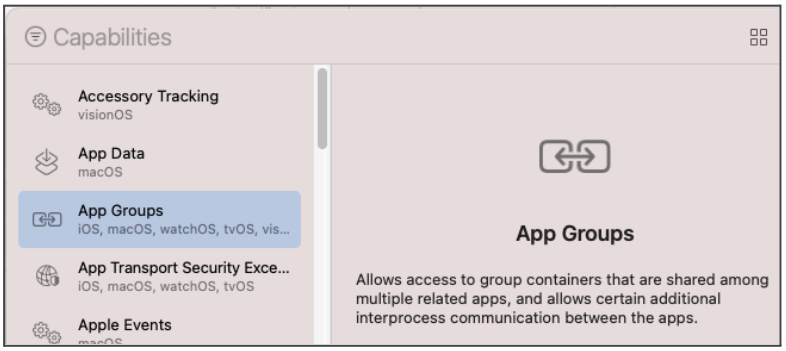
3. Configure App Groups (Shared Secure Storage)

Magic requires a **shared storage container** to synchronize data between the main app and the keyboard.

1. Select **Project → YourApp Target → Signing & Capabilities (All)**



2. Click **+ Capability → App Groups**

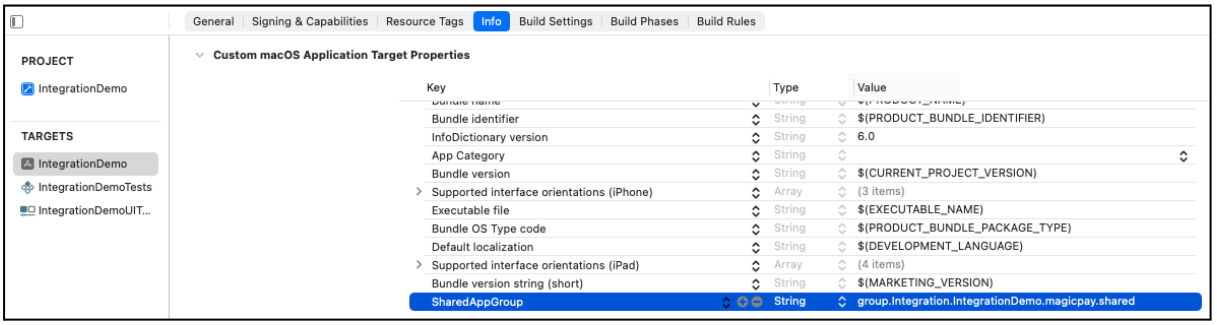


3. Create a new App Group:

```
group.<yourcompany>.<yourapp>.magicpay.shared
```

4. Open **YourApp/Info.plist** → Add:

```
None
<key>SharedAppGroup</key>
<string>group.<yourcompany>.<yourapp>.magicpay.shared</string>
```



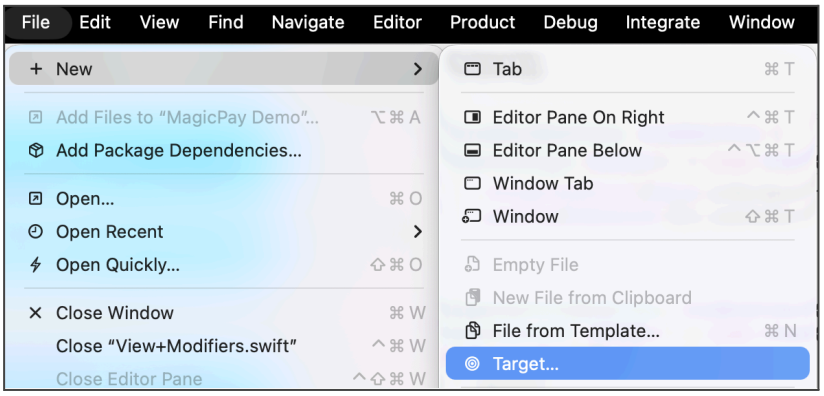
4. Privacy Usage Descriptions (Required)

In **YourApp/Info.plist**, add or adjust messaging to match your UX tone:

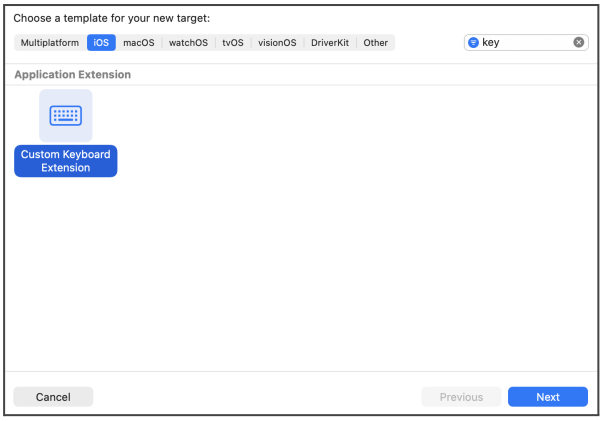
```
None
<key>NSFaceIDUsageDescription</key>
<string>We use Face ID to securely authorize transactions.</string>

<key>NSTouchIDUsageDescription</key>
<string>We use Touch ID to securely authorize transactions.</string>

<key>NSLocationWhenInUseUsageDescription</key>
<string>We use your location to enhance transaction security.</string>
```

5. Create the Magic Keyboard Extension

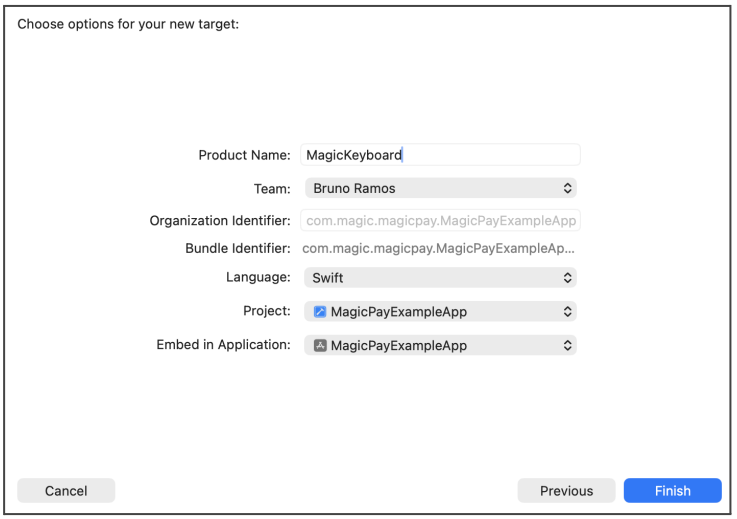
1. **File → New → Target…**



2. Select **Custom Keyboard Extension**



3. **Product Name:** `MagicKeyboard` and **Finish**

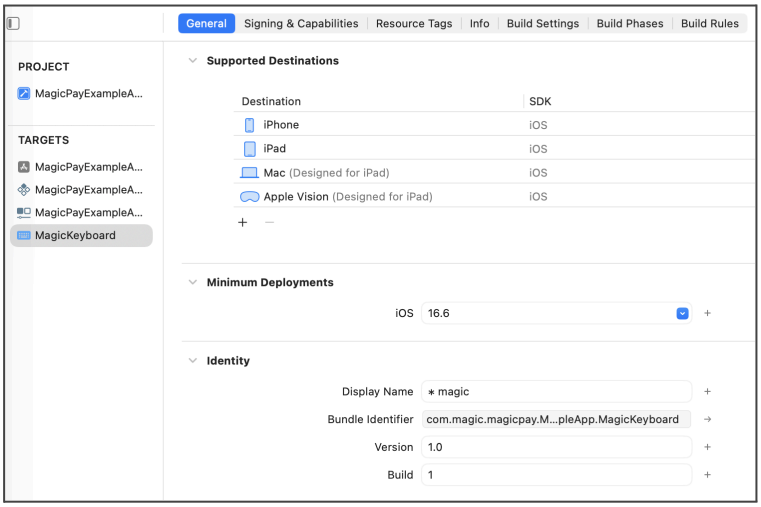


A new folder containing `KeyboardViewController.swift` will be added.

Configure the extension:

1. Select **Target: MagicKeyboard → General**

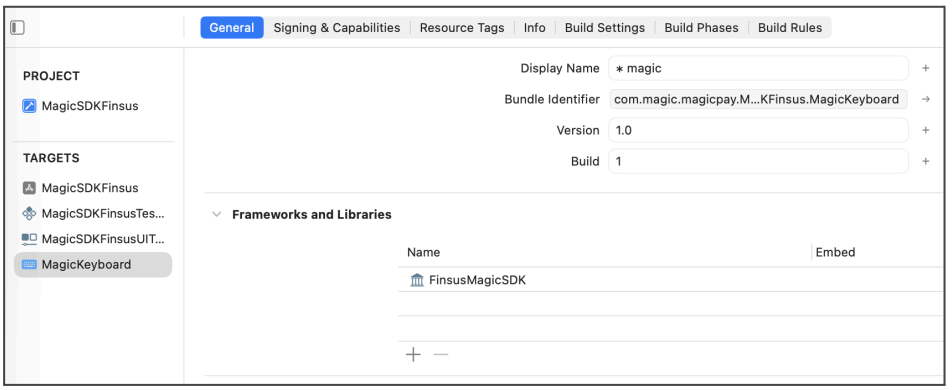
2. **Display Name:** `* magic` *(this is what users will see in iOS keyboard list) with the Main App Name* (`* magic – [App]`)



3. Ensure **Bundle Identifier** contains `magicpay`

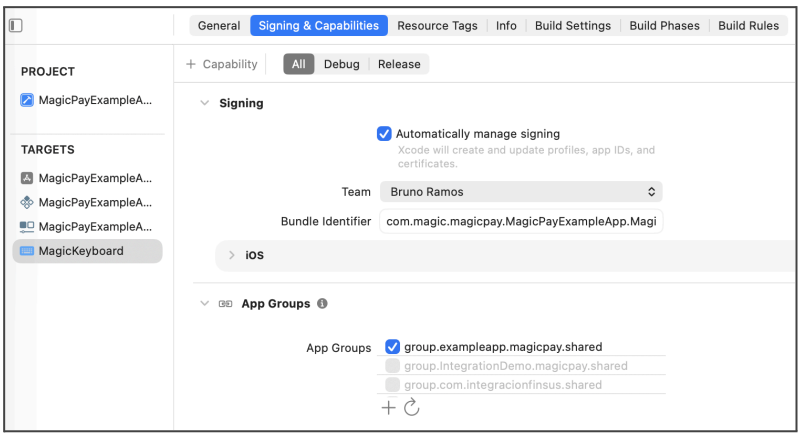
Add Magic SDK to the Keyboard target

1. **Project → Target → YourAppKeyboard → General → Frameworks and Libraries → add the MagicSDK**



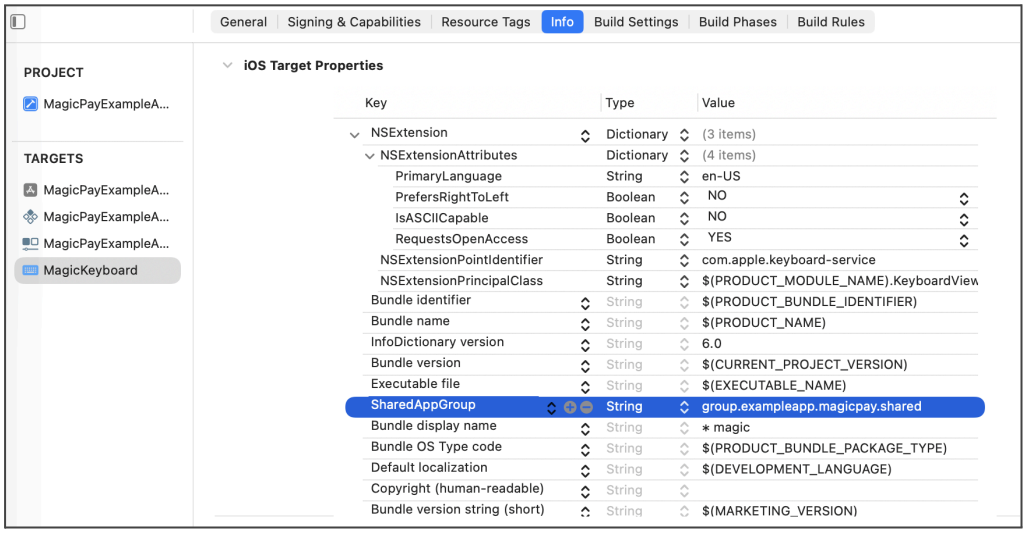
Enable App Group for the Keyboard

1. Select **MagicKeyboard Target → Signing & Capabilities (All)**
2. Add **App Groups**
3. Select the same App Group: `group.<yourcompany>.<yourapp>.magicpay.shared`



4. Open **Keyboard/Info.plist,** add:

```
None
SharedAppGroup = group.<yourcompany>.<yourapp>.magicpay.shared
```



5. In the **Keyboard/Info.plist**, make sure **RequestOpenAccess = YES**
   NSExtension → NSExtensionAttributes → RequestOpenAccess = YES

6. Initialize Magic in Your Application

MagicPaySDK automatically determines whether the user has already completed onboarding. If onboarding is complete, the SDK will show the **Magic Home Screen**.
If not, the SDK will display the **Magic Onboarding Flow**.

Your responsibility is simply to: 1) Provide user/account data and 2) Call `getMainView()` to display Magic.

```C/C++
import MagicPaySDK
import MagicPayCoreSDK
// Datos de cuenta del usuario
let accountList = [
    AccountData(accountId: "ACCOUNT_ID", alias: "ACCOUNT_ALIAS")
]


// Datos de autorización (password y PIN opcionales)
let authorizationData = ClientAuthorizationData(
    password: nil,           // Magic lo solicitará durante onboarding
    pin: nil,                // Magic lo solicitará durante onboarding
    base64PublicKey: nil     // Solo requerido si se proporciona
password/pin
)
// Datos del usuario
let userData = UserData(
    userId: "USER_ID",
    username: "USERNAME",
    accountList: accountList,
    authorizationData: authorizationData
)

// Configuración del ambiente
let envData = EnvironmentData(
    environment: .dev,  // Opciones: .dev o .production
    apiKey: "API_KEY"
)

// Inicializar SDK
self.magicPaySDK = MagicPaySDK(
    environmentData: envData,
    fetchUserData: { return userData }
)
// To display Magic (onboarding or home automatically):
magicPaySDK.getMainView()
```

7. Initialize the Magic Keyboard

This step wires up the custom keyboard in your **Keyboard Extension** target.
Make sure you already:

- Added `magic_ios_sdk` to the **Keyboard** target.
- Enabled the **same App Group** as the main app and set `SharedAppGroup` in the Keyboard **Info.plist**.
- (Optional) Set **Display Name** to `* magic` so users recognize it in iOS.

```C/C++
import UIKit
import SwiftUI
import MagicSDK
import MagicPayCoreSDK

class KeyboardViewController: UIInputViewController {

    override func viewDidLoad() {
        super.viewDidLoad()

        // Configuración del ambiente (mismo API key que la app principal)
        let envData = EnvironmentData(
            environment: .dev,  // Opciones: .dev o .production
            apiKey: "API_KEY"
        )

        // Obtener la vista del teclado de Magic
        let magicKeyboard = MagicSDK.getKeyboardMainView(
            environmentData: envData,
            textDocumentProxy: self.textDocumentProxy,
            inputViewController: self,
            switchKeyboardAction: { [weak self] in
                self?.advanceToNextInputMode()
            }
        )

        // Integrar la vista de SwiftUI en UIKit
        let child = UIHostingController(rootView: magicKeyboard)
        let childView = child.view!
        childView.translatesAutoresizingMaskIntoConstraints = false
        view.addSubview(childView)

        // Configurar constraints
        NSLayoutConstraint.activate([
            childView.topAnchor.constraint(equalTo: view.topAnchor),
            childView.leadingAnchor.constraint(equalTo: view.leadingAnchor),
```

```
            childView.trailingAnchor.constraint(equalTo:
view.trailingAnchor),
            childView.bottomAnchor.constraint(equalTo: view.bottomAnchor),
            childView.heightAnchor.constraint(equalToConstant: 390)
        ])
    }
}
```

9. Validation Checklist

Category	Validation Item	Status
App Groups	App and Keyboard both have the same App Group enabled in **Signing & Capabilities**	Pending
	`SharedAppGroup` key is present in the **App Info.plist** with the correct value	Pending
	`SharedAppGroup` key is present in the **Keyboard Info.plist** with the **exact same value**	Pending
SDK Targets	`magic_ios_sdk` is added to the **Main App target**	Pending
	`magic_ios_sdk` is added to the **Keyboard target**	Pending
Keyboard Target Configuration	Keyboard target **Bundle Identifier** contains `"magicpay"`	Pending
	`RequestsOpenAccess = YES` is included in **Keyboard Info.plist** (only if Full Access is required)	Pending
	`SharedAppGroup` key is present and matches the App's value	Pending
	Keyboard **Display Name** is set to `* magic` (Target → General)	Pending

Backend Integration

Core Integration

New payment link flow

After the creation of a new payment link our system will notify through webhook all the new payment link creation.

- Create a webhook to receive the payment information.

```JSON
{
	"paymentId":"4679b372-e3d4-43f4-a978-ac23a13aeb3c",
	"userId":"fd80789f-213e-452f-a83c-d241f6c1e0ed",
	"accountId": "1eaca391-995c-4760-96d5-58c9f438cf6f",
	"token":"1eaca391-995c-4760-96d5-58c9f438cf6f",
	"amount": 100
}
```

It is important to save `all the values` for the claim payment process.

Claim payment link flow

For this process we need an endpoint to send the request to process the payment link and process the transaction and send the money to the receptor account

```None
{
	"paymentId":"4679b372-e3d4-43f4-a978-ac23a13aeb3c",
	"token":"1eaca391-995c-4760-96d5-58c9f438cf6f",
	"receptorAccount": "012180015532594712",
}
```

To ensure the security of the information we need to encrypt the information with a handshake process to share the key to encrypt the information or through a VPN